

Preguntas y respuestas para defensa tecnica

Proyecto PAMPA / ARINC 429 logico-temporal

Fecha de preparacion: 3 de junio de 2026

Este documento reúne veinte preguntas posibles de un tutor o evaluador técnico sobre el estado actual del proyecto. Las respuestas están pensadas para una explicación oral: técnicas, directas y honestas sobre lo que ya fue validado y lo que todavía pertenece a una fase futura.

1. Cual es el objetivo concreto del proyecto hasta esta fase?

El objetivo fue construir y validar una arquitectura de laboratorio inspirada en ARINC 429, separando la generación, recepción, sniffing, transporte hacia host y visualización. En la fase actual no se declara cumplimiento eléctrico ARINC 429 real; se declara una recreación lógica y temporal a 100 kbps con codificación bipolar lógica con retorno a cero, captura pasiva, ACK inverso, bridge SPI/HTTP y dashboard operativo.

La importancia del resultado es que el sistema completo ya funciona como cadena extremo a extremo: ARINC-TX transmite, ARINC-RX recibe y responde, ARINC-SNIFFER observa ambas direcciones, la Raspberry Pi 3B+ publica el estado y la notebook visualiza y guarda histórico con Prometheus/Grafana.

2. Que significa que la señal sea bipolar con retorno a cero?

En ARINC 429 real, la información se transmite con tres estados eléctricos diferenciales: positivo, negativo y nulo. En este proyecto se recrea esa idea en forma lógica usando dos pines por canal. Para un bit alto se activa una línea, para un bit bajo se activa la otra, y luego ambas vuelven a cero durante la segunda mitad del bit.

Por eso se habla de retorno a cero: cada bit tiene una mitad activa y una mitad en estado nulo. A 100 kbps, cada bit dura 10 µs; en la implementación validada la media celda activa dura aproximadamente 5 µs y la segunda media celda vuelve a cero.

3. Esto ya es ARINC 429 real o todavía es una emulación?

Es una recreación lógica/temporal de ARINC 429, no una interfaz eléctrica ARINC 429 real. Lo validado es la forma temporal, la semántica de palabras, labels, SDI, SSM, paridad, ACK y observabilidad del sistema.

Lo que falta para hablar de interfaz ARINC 429 real es la capa eléctrica: drivers o transceptores adecuados, niveles de línea, protecciones, adaptación de impedancias, conectores, criterios de aislamiento si correspondiera y validación instrumental sobre esa nueva etapa. Esa es una fase futura, no una limitación del cierre actual.

4. Por que los canales se llaman FWD y REV?

FWD significa forward y representa el sentido principal de comunicacion, desde ARINC-TX hacia ARINC-RX. REV significa reverse y representa el sentido inverso, usado por ARINC-RX para enviar ACK hacia ARINC-TX.

El nombre no intenta copiar una nomenclatura oficial de ARINC 429, sino describir la topologia logica del laboratorio. Es util porque evita ambigüedad: FWD es el flujo de datos principal y REV es el retorno de control o confirmacion.

5. Por que se uso un ACK si ARINC 429 normalmente es un bus unidireccional?

El ACK no busca copiar literalmente ARINC 429 avionico clasico. Se agrego como mecanismo de laboratorio para cerrar el lazo funcional, medir recepcion por lotes y validar el canal inverso. En otras palabras, permite comprobar que el receptor no solo recibe palabras, sino que procesa lotes y responde de manera observable.

En la arquitectura queda separado: FWD transporta datos utiles y REV transporta confirmacion. Esto ayudo a validar recepcion, sincronizacion, sniffing bidireccional y robustez de la visualizacion.

6. Que valida exactamente el osciloscopio?

El osciloscopio valida que la forma temporal generada por las Pico coincide con lo esperado para la fase logica: pulsos de aproximadamente 5 us de media celda activa, bit time de 10 us a 100 kbps y retorno a cero en la segunda mitad del bit.

Tambien se valido la reconstruccion diferencial conceptual mediante $MATH = CH1 - CH2$, observando tres estados: positivo, cero y negativo. Esto demuestra la bipolaridad logica de la senal, aunque todavia no valide niveles electricos ARINC 429 reales.

7. Por que se eligio 100 kbps?

100 kbps es una velocidad clasica asociada a ARINC 429 de alta velocidad y permite una referencia clara para la validacion temporal. A esa tasa, el bit time es 10 us, una escala que se puede medir con osciloscopio y que es razonable para generar con PIO en Raspberry Pi Pico.

Elegir 100 kbps tambien permite separar problemas: primero se valida que la capa logica y temporal funcione a una tasa representativa; luego, en una fase electrica, se puede trabajar sobre transceptores y proteccion sin rediseñar todo el software.

8. Que hace cada placa Pico en la arquitectura actual?

ARINC-TX genera las palabras y las transmite por el canal FWD. ARINC-RX escucha FWD, decodifica las palabras y emite ACK por el canal REV. ARINC-SNIFFER observa pasivamente ambos canales, filtra por label y SDI, mantiene snapshots y estadisticas, y expone esa informacion a la Raspberry Pi 3B+ por SPI.

La separacion en tres nodos es importante porque reproduce roles de sistema: emisor, receptor y observador pasivo. Esto hace que el sniffer no sea parte activa del enlace TX/RX y pueda evaluarse como herramienta de monitoreo.

9. Por que el sniffer filtra por label y SDI?

ARINC 429 transporta informacion identificada principalmente por label, y el SDI permite separar fuentes o destinos logicos. Filtrar por label + SDI permite que la Pico sniffer reduzca la informacion a variables utiles antes de enviarla al host.

Esto evita que la Raspberry Pi 3B+ y el dashboard tengan que procesar todo el flujo crudo. El filtrado en la Pico baja carga, simplifica el bridge y hace mas estable la visualizacion.

10. Hay algun cuello de botella actual?

El cuello principal no esta en Flask ni en el dashboard. Lo que mas condiciona el sistema es el intercambio sniffer -> 3B+ por SPI y el patron de polling del bridge. Cuando se forzo el sistema con un perfil mas agresivo, aumento mucho la cantidad de errores SPI sin mejorar realmente la utilidad operativa.

Por eso quedo recomendado un perfil equilibrado: SPI a 800 kHz, delay por byte de 25 us, polling del bridge de 6 ms y refresco de dashboard de 33 ms. Ese perfil dio mas estabilidad y menos errores que el modo de stress.

11. Si TX y RX trabajan a 100 kbps, por que SPI puede ir a 800 kHz?

Son enlaces distintos y cumplen funciones distintas. El enlace TX/RX a 100 kbps representa la comunicacion logica tipo ARINC. El enlace SPI entre sniffer y 3B+ no retransmite la forma de onda bit a bit; transporta snapshots, contadores y datos filtrados.

SPI necesita suficiente margen para consultar al sniffer, leer estructuras, actualizar estadisticas y responder al dashboard sin atrasarse. Por eso puede trabajar a 800 kHz aunque el enlace observado sea de 100 kbps. No hay contradiccion: uno es el enlace observado y el otro es el canal de telemetria del observador.

12. Por que no usar siempre el perfil SPI de stress a 1.2 MHz?

Porque mas rapido no signifiko mejor. El perfil de stress a 1.2 MHz sobrevivio muchas horas, pero acumulo muchos mas spi_errors. En comparaciones largas, el perfil recomendado movio mas palabras utiles con una tasa de errores SPI muchisimo menor.

El criterio de ingenieria elegido fue estabilidad operativa, no velocidad maxima. Para una demostracion y para uso continuo conviene un sistema que mantenga conectividad, sin overflow, sin drops y con errores despreciables.

13. SPI es la mejor opcion entre sniffer y 3B+? Que pasa con UART o I2C?

SPI es una buena opcion porque tiene buena velocidad, baja latencia y una separacion clara entre master y slave. Sin embargo, en la Pico como SPI slave aparecieron desafios practicos de framing, tiempos y lectura estable. Por eso el proyecto termino usando un protocolo byte a byte robusto y parametros conservadores.

UART podria ser mas simple de estabilizar, pero se aleja del objetivo de explorar un enlace host-sniffer por SPI y puede limitar framing o control temporal. I2C no parece la mejor opcion para esta etapa: tiene mas overhead, opera con lineas compartidas, pull-ups, direccionamiento y posibles

problemas de clock stretching. Una evolucion interesante seria un SPI slave implementado por PIO, porque daria mas control del framing.

14. Por que Ethernet directo mejoro la estabilidad frente a Wi-Fi?

La Raspberry Pi 3B+ por Wi-Fi podia introducir jitter, carga variable o condiciones fisicas que afectaban indirectamente la estabilidad del sniffer y del polling. Al usar Ethernet directo entre notebook y 3B+, el trafico operativo queda mas predecible y separado de la red Wi-Fi.

La evidencia practica fue clara: con Ethernet directo, IP fija y perfil recomendado, el sistema se sostuvo durante corridas largas con conectividad estable, sin overflow y con muy pocos errores SPI.

15. Que significa que DRDY no este conectado?

DRDY seria una linea de data ready para avisar al host que hay informacion nueva disponible. En la arquitectura validada no se usa; el bridge trabaja por polling, consultando al sniffer de forma periodica.

Esto no invalida la fase actual, porque los resultados demuestran que el polling elegido alcanza para sostener la operacion. DRDY queda como mejora futura para reducir polling inutil y hacer el sistema mas orientado a eventos.

16. Como se interpreta la corrida larga con mas de 27 millones de palabras?

La corrida reporto aproximadamente 27,5 millones de palabras aceptadas, cero errores de paridad, cero overflow, cero drops SPI, cero evictions de slots y solo 36 spi_errors. Eso representa una tasa de error extremadamente baja para una corrida de varias horas.

Tambien se mantuvieron vivas las variables TEMPERATURA, VELOCIDAD, ALTITUD y ACK_BATCH, todas con paridad OK y SSM NORMAL. La lectura correcta es que el sistema ya no es una prueba breve: es una baseline estable para mostrar.

17. Que limitaciones conocidas quedan en el sniffer?

Actualmente el snapshot del sniffer tiene 16 slots y el filtro SPI tiene 10 entradas. Para el conjunto actual de variables alcanza: temperatura, velocidad, altitud y ACK_BATCH ocupan solo 4 slots.

Si en el futuro se quiere monitorear muchas mas variables simultaneas, habra que aumentar LATEST_SLOT_CAPACITY y posiblemente SNIFFER_SPI_FILTER_CAPACITY. Esa es una mejora de escala, no un bloqueo de la demostracion actual.

18. Ya esta encarado el diseno de PCB?

Todavia no como diseno final. El proyecto esta en una etapa correcta para empezar a pensar la PCB, pero antes conviene cerrar la decision de la capa electrica real: transceptores o drivers ARINC 429, protecciones, conectores, alimentacion, referencia de masa, aislamiento si aplica y puntos de test.

Hacer una PCB antes de decidir esa interfaz podria fijar errores temprano. La fase actual sirve como baseline funcional; la PCB deberia venir despues de elegir y validar el front-end electrico.

19. Se puede automatizar parte del desarrollo de PCB con Codex?

Si, pero con limites claros. Codex puede ayudar a preparar requisitos, checklist de diseno, BOM preliminar, estructura de proyecto KiCad, scripts para revisar netlists, documentacion de pines, tablas de conectividad y comparacion de alternativas de transceptores.

No conviene delegar ciegamente el diseno fisico completo. La revision electrica, reglas de seguridad, layout de senales, protecciones y validacion instrumental deben pasar por software especializado como KiCad y por revision humana. Codex puede acelerar y ordenar, no reemplazar la validacion de ingenieria.

20. Que se puede afirmar con seguridad ante el tutor?

Se puede afirmar que la fase arinc429_logic quedo validada como recreacion logica/temporal estable a 100 kbps, con ACK reverso, sniffer pasivo, filtrado por label + SDI, bridge SPI/HTTP, dashboard Flask, Prometheus/Grafana y evidencia de osciloscopio.

Tambien se puede afirmar que la arquitectura modular fue correcta: se pudo cambiar la forma de generacion/captura de la senal sin rediseñar la 3B+, el bridge ni el dashboard. Lo que sigue ya no es corregir esta fase, sino avanzar hacia la interfaz electrica ARINC 429 real y, despues, hacia una PCB.